PRN232 – Practical Exam – Paper No 7

PRN232 – Practical Exam – Paper No: 7

Duration: 90 minutes — **Total:** 10 points

INSTRUCTIONS

Please read the instructions carefully before doing the questions.

- You can use materials in your computer, notebook and text book.
- You are **NOT allowed** to use any device to share data with others.

Beside the above conditions, students must follow the following requirements:

1. **The work must complete by using Visual Studio 2022++**
2. **The Framework must be .NET 8.0**
3. **THIS PART IS VERY IMPORTANT, PLEASE READ IT CAREFULLY AND FOLLOW THE INSTRUCTIONS.**
    - You are given a database script (.sql file) in Zip file. Execute the script before doing questions.
    - **You must use the given solution.**
    - **You are not allowed to add any more libraries via NuGet Package Manager into given solution.**
    - **Submission Guideline:** Submit your work for each question separately. For each question, please:
        - Publish your project using the command:

          ```
          dotnet publish -c Release -o ./[QuestionNumber_StudentAccount]
          ```

          Example:

          ```
          dotnet publish -c Release -o ./Q1_trungnthe123432
          ```

        - Submit the root folder of the project into the PEA_Client application.
        - If the root folder of the project is too large, you may delete the following subfolders to reduce its size before submitting: `/bin`, `/obj`

Just one of above requirements is violated, your work will be considered as invalid.

Question 1: (5 points)

You are provided with a database as following diagram:

Database Schema (database.sql — database name: PE_PRN_26SP_P7)

Table	Column	Data Type / Notes
Suppliers	SupplierID	INT – PK, auto-increment
	SupplierName	NVARCHAR(100) NOT NULL
	Specialty	NVARCHAR(200) – supplier's specialty area
	ContractDate	DATE
Products	ProductID	INT – PK, auto-increment
	ProductName	NVARCHAR(200) NOT NULL
	Price	INT – unit price (USD)
	Category	NVARCHAR(100)
ProductSuppliers	ProductID	INT – FK → Products, part of composite PK
	SupplierID	INT – FK → Suppliers, part of composite PK
	SupplyDate	DATE DEFAULT GETDATE()
Batches	BatchID	INT – PK, auto-increment
	ProductID	INT – FK → Products
	WarehouseCode	NVARCHAR(20)
	Quarter	NVARCHAR(20) – e.g., Q1-2024
	Quantity	INT
Customers	CustomerID	INT – PK, auto-increment
	CustomerName	NVARCHAR(100) NOT NULL
	Email	NVARCHAR(100)
	DateOfBirth	DATE
Orders	OrderID	INT – PK, auto-increment
	CustomerID	INT – FK → Customers
	BatchID	INT – FK → Batches
	OrderDate	DATE
	Rating	FLOAT – NULL = the customer has not rated the order

Your task is to develop a **.NET 8 Web API** project (use the given solution — project **Q1**, which already contains the Entity Framework Core models scaffolded from the database) that implements the

following endpoints.

Notes:

- The database connection string must be stored in `appsettings.json` with the key `"MyCnn"`. **Zero marks** if it is stored anywhere else.
- All JSON property names in responses are in **camelCase** (default behavior).
- Examples below assume the API runs at `http://localhost:5000`.

1. GET /api/customers

Retrieve a list of all customers, supporting both **JSON** and **XML** formats, and calculate their average rating.

- Return a list of customers with the following information: **CustomerId**, **CustomerName**, **Email**, and **AvgRating**.
- **AvgRating** is calculated as the average of all **Rating** values in the **Orders** table for that customer. Records with a NULL Rating should be **excluded** from the calculation.
- Support response formats in both **JSON** and **XML** using media formatters.
- Example:

```
GET http://localhost:5000/api/customers
HTTP 200
[
  {
    "customerId": 1,
    "customerName": "Nguyen Hoang Long",
    "email": "longnh@shop.com.vn",
    "avgRating": 4.25
  },
  {
    "customerId": 2,
    "customerName": "Vu Minh Anh",
    "email": "anhvm@shop.com.vn",
    "avgRating": 3.5
  },
  ...
]
```

2. GET /api/customer-loyalty

Retrieve a list of customers similar to /api/customers but with filtering and enhanced pagination features.

- **Filtering:** Use query parameters:
  - `minRating`: Only return customers whose AvgRating is greater than or equal to this value (if provided).
  - `customerName`: Search for customers whose name contains this string (if provided).
- **Detailed pagination:** Use query parameters:
  - `page`: Current page (default is 1).
  - `pageSize`: Number of customers per page (default is 10).
- Return the list of customers for the requested page, along with pagination metadata: **TotalCustomers**, **TotalPages**, **CurrentPage**, and **PageSize**.
- **Error handling:** If page or pageSize is non-positive, return a **400 Bad Request** error with the body `Invalid pagination parameters.`
- Example:

```
GET http://localhost:5000/api/customer-loyalty?
minRating=4&page=-1&pageSize=10
HTTP 400
Invalid pagination parameters.
```

```
GET http://localhost:5000/api/customer-loyalty?
minRating=4&page=1&pageSize=10
HTTP 200
{
  "data": [
    {
      "customerId": 1,
      "customerName": "Nguyen Hoang Long",
      "email": "longnh@shop.com.vn",
      "avgRating": 4.25
    },
    {
      "customerId": 3,
      "customerName": "Le Bao Chau",
      "email": "chaulb@shop.com.vn",
      "avgRating": 4.5
    }
  ],
```

```
  "totalCustomers": 2,
  "totalPages": 1,
  "currentPage": 1,
  "pageSize": 10
}
```

3. PUT /api/orders/{orderId}/rating

Update the rating for an existing order.

- Accept a **Rating** value (float) from the request body:

```
{ "rating": 4 }
```

- **Logic:**
  - If the **Rating** value is not within the range [0, 5], return a **400 Bad Request** with the message "Rating must be between 0 and 5".
  - If the **OrderId** does not exist, return a **404 Not Found** error.
- Return the updated order details, including **OrderId**, **CustomerId**, and the new **Rating**.
- Example:

```
PUT http://localhost:5000/api/orders/8/rating
Body: { "rating": 4 }
HTTP 200
{
  "orderId": 8,
  "customerId": 3,
  "rating": 4
}
```

4. DELETE /api/orders/{orderId}

Cancel a customer's order (Remove the record from the **Orders** table).

- Delete the order with the specified **OrderId** from the URL and return **204 No Content** if success.
- **Error handling:**
  - If the order record already has a rating (**Rating** is not NULL), return a **400 Bad Request** error with the message "Cannot cancel an order that has already been rated".
  - If the **OrderId** does not exist, return a **404 Not Found** error with the message "No order found with provided OrderId".

- Example:

```
DELETE http://localhost:5000/api/orders/9
HTTP 400
Cannot cancel an order that has already been rated
```

Question 2: (5 points)

In this question, you are asked to write an MVC/Razor Pages application. The application fetch data by calling pre-existing RESTful API hosted at **GivenAPIBaseUrl**. The API are provided in a separate project named **GivenAPIs**, which students must run locally to start the API server.

1. Important Notes

- Students **MUST** use **HttpClient** to make calls to the API.
- The value of **GivenAPIBaseUrl** must be written in `appsettings.json` as:

```
{ "GivenAPIBaseUrl": "http://localhost:5100" }
```

- **"GivenAPIBaseUrl"** is a provided key, and students are not allowed to modify it.
- Students get the **GivenAPIBaseUrl** value from **appsettings.json**, combine it with the **endpoint** to call the API.
- When concatenating the base URL with the endpoint, students must explicitly use **string concatenation** (e.g., `"baseUrl" + "/endpoint"`).
- All input and output elements in the HTML source **must** have an **'id'** attribute to ensure accessibility and traceability (Students can refer to the **list.html** and **detail.html** files in given materials, which provides code snippets for the assignment).

2. Provided APIs

- **GET /api/suppliers/search?name={name}&specialty={specialty}**: Returns a list of suppliers filtered by their Supplier Name and Specialty. In case, {name}, {specialty} is missing, return all **suppliers**.
- **GET /api/specialties**: Returns a list of all unique specialty areas.
- **GET /api/suppliers/{supplierId}**: Returns detailed information about a specific supplier including their supplied products.

3. Requirements

3.1. Supplier List Page (see list.html)

URL: /Supplier

- **Search Form:**
  - An input field for Supplier Name (id: `ip_supplierName`).
  - An input field for Specialty (id: `ip_specialty`).
  - A Search button (id: `bt_search`).
- **Table Display**: Display data in a tabular format with columns:
  - **Supplier Name**.
  - **Specialty**.
  - **Contract Date**.
  - **Total Products** (Number of products supplied).
- Each row in the table should have a **"View Products"** link redirecting to the detail page with the URL `/Supplier/{SupplierId}`.

ID Requirements:

- Each `<td>` tag in the table: `td_{columnName}_{supplierId}`. Example: `td_supplierName_1`, `td_specialty_1`.
- Each "View Products" link must be placed in an `<a>` tag with id `a_{supplierId}`. Example: `a_1`

3.2. Filter logic

- When a user enters criteria and clicks the "Search" button, the list should only show suppliers matching both the Name and Specialty.
- If both fields are empty, list all suppliers.

3.3. Supplier Detail Page (see detail.html)

URL: /Supplier/{SupplierId}

- **Display basic information** of the supplier: **SupplierID**, **SupplierName**, **Specialty**.
- **Display a list of all products** supplied by this supplier in a table with columns: **ProductID**, **ProductName**, **Price**.

ID Requirement:

- Basic info fields in `<span>` tags: `span_{supplierId}`, `span_{supplierName}`, `span_{specialty}`. Example: `span_1`, `span_Cong ty TNHH Alpha`
- Each `<td>` tag in the product table: `td_{columnName}_{productId}`. Example: `td_productID_1`, `td_productName_1`

4. Summary of HTML Elements ID

The HTML id requirements are summarized in the table below:

Page	Element	Tag	Id
/Supplier	Input Supplier Name	`<input>`	`ip_supplierName`
	Input Specialty	`<input>`	`ip_specialty`
	Button Search	`<input>`	`bt_search`
	Each cell in the table	`<td>`	`td_{columnName}_{supplierId}`
	Each View Products link	`<a>`	`a_{supplierId}`
/Supplier/{id}	Field SupplierID	`<span>`	`span_{supplierId}`
	Field SupplierName	`<span>`	`span_{supplierName}`
	Field Specialty	`<span>`	`span_{specialty}`
	Each cell in product table	`<td>`	`td_{columnName}_{productId}`

See **list.html** and **detail.html** in the Given Materials.

Note: Ensure all ID requirements are strictly followed so that the examiner can automatically verify your work.

5. Summary of Required URLs

Function	URL
Search and list suppliers	/Supplier
Details of the supplier	/Supplier/{SupplierId}

--- END OF PAPER ---